BearShare: A Secure File-Sharing Platform

Paige Brathwaite & Smila Gala-Alfonso

Table of Contents

01 Project Overview

02 Security Architecture

03 Key Security Implementations

04 Challenges & Lessons Learned

Project Overview

BearShare is a bear-themed secure document sharing web application for uploading, storing, viewing, downloading, and sharing confidential files.

- The site uses **RBAC**, with **system roles like admin, user, and guest**, and **document permissions roles** such as **owner, editor, and viewer**.

- To bolster security, **password hashing utilizes bcrypt, and the site features encrypted file storage, session expiration, input validation, security headers, and audit logging**.

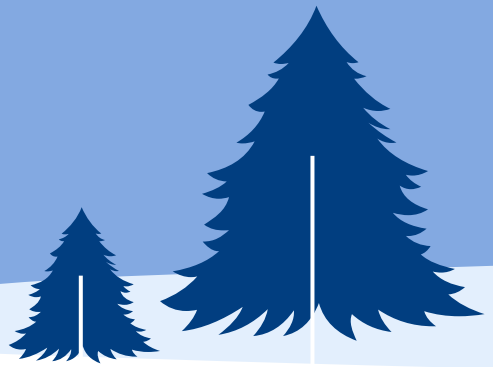
The goal of the project was to build a file-sharing system that is both usable and secure.

Key Security Features

1. Secure Authentication: Passwords are hashed using bcrypt, with **password complexity rules, account lockout after repeated failures, and IP-based rate limiting**

2. Role-Based Access Control: BearShare enforces both system roles (**admin, user, guest**) and document roles (**owner, editor, viewer**) to control who can access and modify files.

3. Encrypted File Storage: Uploaded files are **encrypted** before being stored on disk to **protect sensitive data at rest.**



Key Security Features cont.

4. Session Security: Sessions use **secure random tokens, timeout after inactivity**, and are protected with cookie settings like **HttpOnly and SameSite.**

5. Input Validation: User inputs and file uploads are **validated to reduce risks such as invalid data, malicious uploads, and unsafe file handling.**

6. Security Headers and Logging: The app uses headers like **Content Security Policy** and records security-relevant events such as **failed logins, denied access, uploads, deletes, and role changes.**

Challenges Learned

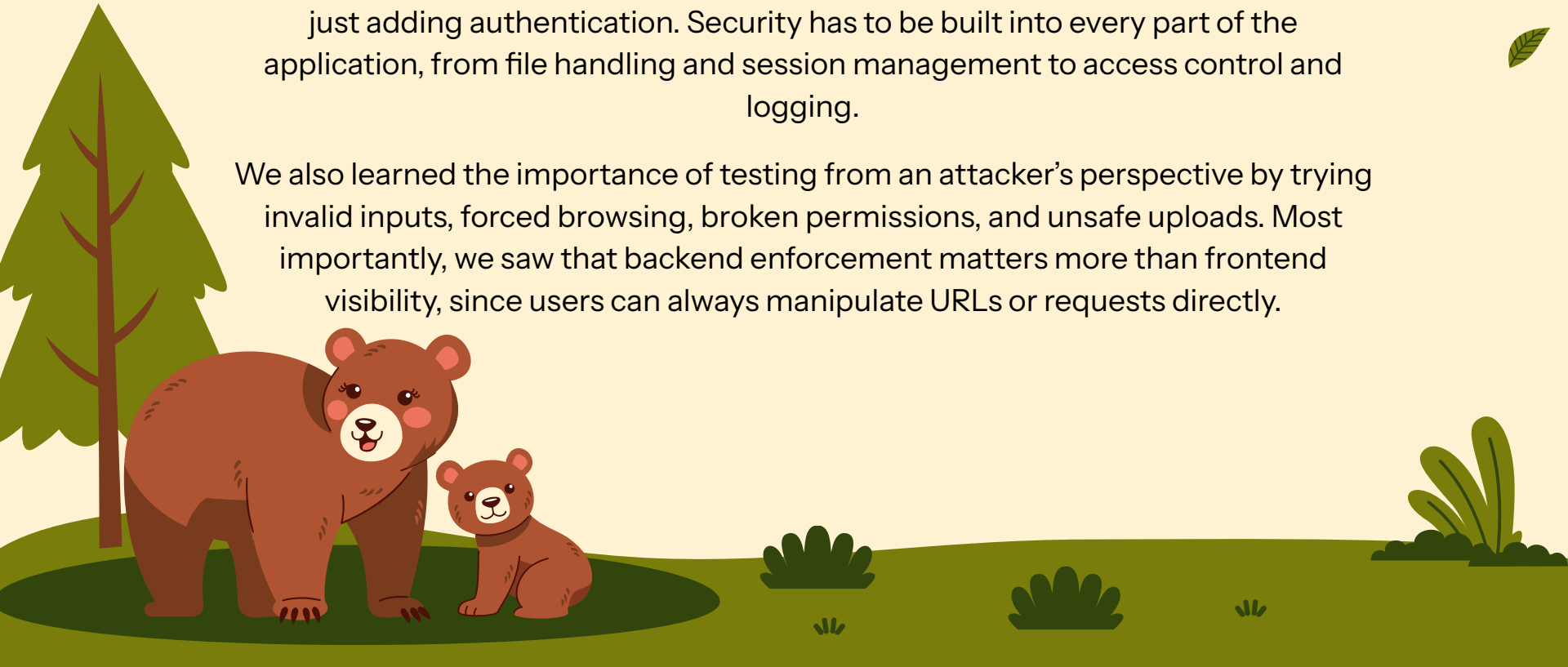
One of the biggest challenges in building BearShare was **balancing security with functionality**. Since the application needed to support uploads, sharing, role-based permissions, versioning, and admin controls, each new feature **introduced more opportunities for access control mistakes or unsafe input handling.**

Another challenge was making sure security protections worked together cleanly, such as account lockout, IP-based rate limiting, encrypted file storage, session expiration, and file validation. It was also sometimes difficult to test and debug features like HTTPS, session handling, and permission changes without slowing down development.

Lessons Learned

This project showed us that secure web development requires much more than just adding authentication. Security has to be built into every part of the application, from file handling and session management to access control and logging.

We also learned the importance of testing from an attacker's perspective by trying invalid inputs, forced browsing, broken permissions, and unsafe uploads. Most importantly, we saw that backend enforcement matters more than frontend visibility, since users can always manipulate URLs or requests directly.



Thanks!

Let us know if you have any questions.

Paige Brathwaite & Smila Gala-Alfonso